

ui.icon 全面详细阐述

`ui.icon` 是 NiceGUI 框架中用于**图标展示与交互**的核心组件，基于 Quasar Framework 的 `QIcon` 组件封装，整合了 Material Icons、Font Awesome 等主流图标库，支持图标样式定制、交互事件绑定、响应式控制等能力。该组件轻量化、易用性强，是构建 UI 界面（按钮、导航栏、状态提示等）的基础元素，广泛应用于各类 Web 应用场景。以下从核心特性、使用场景、配置选项、事件系统、进阶用法等方面展开详细说明。

一、核心概述

1. 功能定位

- 基础能力：加载并展示各类图标，支持主流图标库的图标引用；
- 样式定制：支持尺寸、颜色、旋转、动画、阴影等样式自定义；
- 交互支持：可绑定点击、悬停等事件，实现图标按钮、开关等交互功能；
- 响应式控制：支持通过响应式变量动态切换图标、样式、可见性；
- 扩展能力：可嵌套在其他组件（如 `ui.button`、`ui.card`）中，或结合叠加层实现复杂效果。

2. 核心优势

优势点	说明
多图标库兼容	原生支持 Material Icons（默认）、Font Awesome，可扩展自定义图标库；
轻量化集成	无需额外引入图标库文件（NiceGUI 自动加载所需资源），减少网络请求；
样式灵活度高	支持通过 Quasar/Tailwind 类或内联样式，快速定制图标外观；
交互能力完备	支持点击、双击、悬停等事件，可直接作为交互元素使用；
响应式适配	可绑定响应式变量，动态切换图标、颜色、尺寸，适配不同状态；
生态兼容性强	可与 <code>ui.button</code> 、 <code>ui.menu</code> 、 <code>ui.badge</code> 等组件无缝联动，构建统一 UI 风格。

3. 支持的图标库与引用方式

NiceGUI 内置 3 类核心图标库，同时支持自定义图标，引用方式简洁直观：

图标库	引用格式	示例	说明
Material Icons（默认）	直接使用图标名称（无需前缀）	<code>ui.icon('home')</code>	官方图标库： https://font.s.google.com/icons ，免费开源，覆盖主流场景；
Material Icons Outlined	图标名称前缀 <code>outlined_</code>	<code>ui.icon('outlined_home')</code>	Material Icons 的轮廓版，风格更轻盈；

图标库	引用格式	示例	说明
Font Awesome	图标名称前缀 fa- (需确保加载 Font Awesome 资源)	<code>ui.icon('fa-home')</code>	第三方图标库: https://fontawesome.com/ , 需手动加载免费 / 付费资源;
自定义图标 (本地 / SVG)	使用本地图标文件路径 或 SVG 字符串	<code>ui.icon('static/icons/my-icon.svg')</code>	支持 SVG、PNG 等格式, 适用于品牌图标、自定义图标;

注: Font Awesome 需手动加载资源 (见 “进阶用法 - 自定义图标库”), 默认不加载以减少资源体积。

二、基础使用与图标引用

`ui.icon` 的核心参数为 `name` (图标名称 / 路径), 用法简洁, 以下是不同场景的基础使用示例:

1. 基础图标展示 (默认 Material Icons)

直接引用 Material Icons 的图标名称, 快速展示图标:

```
from nicegui import ui

# 基础图标 (默认尺寸、颜色)
ui.icon('home') # 主页图标
ui.icon('search') # 搜索图标
ui.icon('settings') # 设置图标

# 轮廓版图标 (Material Icons Outlined)
ui.icon('outlined_home')
ui.icon('outlined_search')

ui.run()
```

2. Font Awesome 图标使用

需先加载 Font Awesome 资源, 再通过 `fa-` 前缀引用图标:

```
from nicegui import ui

# 加载 Font Awesome 免费版资源 (通过 CDN)
ui.add_head_html('<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.1/css/all.min.css">')

# 引用 Font Awesome 图标 (前缀 fa-)
ui.icon('fa-home') # 主页图标
ui.icon('fa-search') # 搜索图标
ui.icon('fa-cog') # 设置图标 (与 Material Icons 名称可能不同)
ui.icon('fa-solid fa-user') # 指定实心样式 (Font Awesome 支持 solid/regular/light 等)

ui.run()
```

3. 自定义本地图标 (SVG/PNG)

引用本地图标文件（推荐 SVG 格式，支持矢量缩放无失真）：

```
from nicegui import ui

# 引用本地 SVG 图标（相对路径，建议放在 static 文件夹）
ui.icon('static/icons/brand-logo.svg').classes('w-10 h-10')

# 引用本地 PNG 图标（需指定尺寸，避免拉伸）
ui.icon('static/icons/notification.png').classes('w-8 h-8 object-contain')

ui.run()
```

- **路径说明：**本地图标文件建议放在 `static` 文件夹（NiceGUI 默认静态资源目录），引用时直接使用 `static/[文件夹]/[文件名]`；
- **格式建议：**优先使用 SVG 格式（矢量图，支持任意尺寸缩放），PNG 格式需注意分辨率，避免模糊。

4. 响应式图标切换

结合 NiceGUI 的响应式变量，动态切换图标（如切换“播放 / 暂停”图标）：

```
from nicegui import ui

# 响应式变量控制图标状态
is_playing = ui.reactive(False)

# 图标绑定响应式变量（根据 is_playing 切换图标）
icon = ui.icon().bind_name_from(
    is_playing,
    lambda playing: 'pause' if playing else 'play_arrow'
).classes('text-2xl text-blue-500 cursor-pointer')

# 点击切换图标状态
icon.on('click', lambda: setattr(is_playing, 'value', not is_playing.value))

ui.run()
```

三、核心配置选项

`ui.icon` 提供丰富的配置属性，用于控制图标样式、行为、响应式状态等，核心属性如下（继承自 `ui.element`，并新增图标专属配置）：

属性名	类型	说明	默认值		
name	BindableProperty	图标名称 / 路径（支持 Material Icons、Font Awesome、本地文件），可绑定响应式变量；	None		

属性名	类型	说明	默认值		
size	`str`	int	None`	图标尺寸（支持数字（px）、Quasar 尺寸类（sm/md/lg）、Tailwind 类）；	None
color	`str`	None`	图标颜色（支持颜色名称、十六进制、RGB，可绑定响应式变量）；	None	
rotate	int	图标旋转角度（0~360，正数顺时针，负数逆时针）；	0		
flip	`str`	None`	图标翻转：'horizontal'（水平翻转）、'vertical'（垂直翻转）、'both'（双向翻转）；	None	
animation	`str`	None`	图标动画：'spin'（旋转）、'pulse'（脉冲），可结合 animation_duration 控制时长；	None	
animation_duration	`str`	int`	动画时长（如 '2s'、500（ms）），仅当 animation 生效时可用；	'1s'	
classes	Classes[Self]	应用于图标元素的 CSS 类（支持 Tailwind、Quasar 类）；	''		
style	Style[Self]	内联 CSS 样式（如 'cursor: pointer;'）；	''		
visible	BindableProperty	组件是否可见（可绑定响应式变量）；	True		
tooltip	`str`	None`	鼠标悬停时显示的提示文本（v2.16.0+ 支持）；	None	

配置示例（自定义图标样式与行为）

```
from nicegui import ui

# 1. 自定义尺寸、颜色、旋转
ui.icon('home', size=32, color='red').classes('mr-4') # 32px 红色主页图标

# 2. 翻转 + 动画（旋转）
ui.icon('refresh', flip='horizontal', animation='spin',
animation_duration=2000).classes('mr-4 text-blue-500') # 水平翻转 + 2秒旋转动画

# 3. Quasar 尺寸类 + 阴影
ui.icon('settings', size='lg', color='#007bff').classes('mr-4 shadow-md') # 大号（lg）蓝色设置图标 + 阴影

# 4. Tailwind 类定制（更灵活）
ui.icon('alert', classes='text-2xl text-yellow-500 rotate-45') # 2xl 尺寸 + 黄色 + 45度旋转

# 5. 带提示文本的图标
ui.icon('help', tooltip='点击获取帮助').classes('text-xl text-green-500 cursor-pointer')

ui.run()
```

四、核心事件系统

`ui.icon` 支持完整的鼠标事件，可作为交互元素（如图标按钮、开关）使用，事件回调中可获取事件对象（坐标、按键等信息）。

1. 支持的事件类型与参数

事件名	触发时机	事件参数 (e) 关键属性
<code>click</code>	鼠标左键点击图标时触发	<code>client_x/client_y</code> : 屏幕坐标; <code>target</code> : 事件目标元素;
<code>dblclick</code>	鼠标左键双击图标时触发	同 <code>click</code>
<code>mousedown</code>	鼠标按下（任意按键）时触发	同 <code>click</code> + <code>button</code> : 鼠标按键 (0 = 左键, 1 = 中键, 2 = 右键);
<code>mouseup</code>	鼠标松开（任意按键）时触发	同 <code>mousedown</code>
<code>mouseenter</code>	鼠标进入图标区域时触发	同 <code>click</code>
<code>mouseleave</code>	鼠标离开图标区域时触发	同 <code>click</code>
<code>contextmenu</code>	鼠标右键点击（触发上下文菜单）时触发	同 <code>click</code> (可用于自定义右键菜单)

2. 事件绑定示例（核心场景）

示例 1：图标按钮（点击事件）

```
from nicegui import ui

# 点击图标触发通知
ui.icon('notifications', classes='text-xl text-red-500 cursor-pointer').on('click', lambda: ui.notify('收到新通知!'))

# 带悬停效果的图标按钮
ui.icon('delete', classes='text-xl text-gray-500 cursor-pointer transition-colors hover:text-red-500').on('click', lambda: ui.notify('删除操作!', type='warning'))

ui.run()
```

示例 2：图标开关（双击切换状态）

```
from nicegui import ui

# 响应式变量控制开关状态
is_enabled = ui.reactive(True)

# 双击图标切换状态（颜色 + 图标）
icon = ui.icon(
```

```

        'check_circle' if is_enabled.value else 'cancel',
        color='green' if is_enabled.value else 'red',
        classes='text-2xl cursor-pointer'
    )

    # 绑定响应式变量（动态更新图标和颜色）
    icon.bind_name_from(is_enabled, lambda enabled: 'check_circle' if enabled else 'cancel')
    icon.bind_color_from(is_enabled, lambda enabled: 'green' if enabled else 'red')

    # 双击事件切换状态
    icon.on('dblclick', lambda: setattr(is_enabled, 'value', not is_enabled.value))

    # 显示状态文本
    ui.label().bind_text_from(is_enabled, lambda enabled: '状态: 启用' if enabled else '状态: 禁用')

    ui.run()

```

示例 3：自定义右键菜单（contextmenu 事件）

```

from nicegui import ui

# 图标（右键触发菜单）
icon = ui.icon('more_vert', classes='text-xl cursor-pointer')

# 右键菜单（初始隐藏）
menu = ui.menu(icon).classes('w-48')
with menu:
    ui.menu_item('编辑', on_click=lambda: ui.notify('编辑图标'))
    ui.menu_item('删除', on_click=lambda: ui.notify('删除图标', type='warning'))
    ui.separator()
    ui.menu_item('查看详情', on_click=lambda: ui.notify('图标详情'))

# 禁用默认右键菜单（仅保留自定义菜单）
icon.on('contextmenu', lambda e: e.preventDefault())

ui.run()

```

五、进阶用法（核心场景落地）

场景 1：图标按钮组合（与 ui.button 联动）

`ui.icon` 常与 `ui.button` 结合，构建图标按钮（带图标的按钮），是 UI 界面的常用元素：

```

from nicegui import ui

# 1. 图标 + 文本按钮
ui.button('首页', icon='home', on_click=lambda: ui.notify('进入首页')).classes('mr-2')

# 2. 仅图标按钮（无文本）
ui.button(icon='search', on_click=lambda: ui.notify('搜索')).classes('mr-2')

```

```

# 3. 带颜色的图标按钮
ui.button(icon='add', color='green', on_click=lambda: ui.notify('添加')).classes('mr-2')

# 4. 圆形图标按钮
ui.button(icon='refresh', rounded=True, on_click=lambda: ui.notify('刷新')).classes('mr-2')

# 5. 响应式图标按钮（动态切换图标）
is_playing = ui.reactive(False)
play_btn = ui.button(
    icon='pause' if is_playing.value else 'play_arrow',
    on_click=lambda: setattr(is_playing, 'value', not is_playing.value)
)
play_btn.bind_icon_from(is_playing, lambda playing: 'pause' if playing else 'play_arrow')

ui.run()

```

场景 2：图标导航栏（与 ui.menu 联动）

构建侧边导航栏，使用图标提升视觉体验：

```

from nicegui import ui

# 侧边导航栏容器
with ui.column().classes('w-16 h-screen bg-gray-100 p-4 items-center gap-6'):
    # 导航图标（带提示文本）
    ui.icon('home', tooltip='首页').classes('text-xl text-gray-700 cursor-pointer hover:text-blue-500')
    ui.icon('search', tooltip='搜索').classes('text-xl text-gray-700 cursor-pointer hover:text-blue-500')
    ui.icon('library_books', tooltip='知识库').classes('text-xl text-gray-700 cursor-pointer hover:text-blue-500')
    ui.icon('settings', tooltip='设置').classes('text-xl text-gray-700 cursor-pointer hover:text-blue-500')
    ui.separator().classes('w-8')
    ui.icon('account_circle', tooltip='个人中心').classes('text-xl text-gray-700 cursor-pointer hover:text-blue-500')
    ui.icon('logout', tooltip='退出登录').classes('text-xl text-red-500 cursor-pointer hover:text-red-700')

# 主内容区（示例）
with ui.column().classes('ml-20 mt-10'):
    ui.label('主内容区').classes('text-2xl font-bold')

ui.run()

```

场景 3：状态图标（与响应式变量联动）

展示系统状态（如在线 / 离线、加载中、成功 / 失败），通过图标动态反馈状态变化：

```
from nicegui import ui
import time

# 响应式变量控制状态
status = ui.reactive('loading') # loading/success/error

# 状态图标（动态更新）
status_icon = ui.icon(
    'refresh',
    color='gray',
    animation='spin',
    classes='text-2xl'
)

# 绑定响应式变量（更新图标、颜色、动画）
def update_icon(current_status):
    if current_status == 'loading':
        return {'name': 'refresh', 'color': 'gray', 'animation': 'spin'}
    elif current_status == 'success':
        return {'name': 'check_circle', 'color': 'green', 'animation': None}
    elif current_status == 'error':
        return {'name': 'error', 'color': 'red', 'animation': None}

status_icon.bind_name_from(status, lambda s: update_icon(s)['name'])
status_icon.bind_color_from(status, lambda s: update_icon(s)['color'])
status_icon.bind_prop('animation', status, lambda s: update_icon(s)['animation'])

# 状态文本
status_label = ui.label().bind_text_from(
    status,
    lambda s: {'loading': '加载中...', 'success': '操作成功!', 'error': '操作失败!'}[s]
)

# 模拟状态变化
def simulate_status_change():
    time.sleep(2)
    status.set_value('success')
    time.sleep(2)
    status.set_value('error')

ui.timer(0, simulate_status_change, once=True)

ui.run()
```


场景 4：自定义图标库（加载本地 SVG 图标集）

当需要使用品牌图标、自定义图标时，可加载本地 SVG 图标集（如 Iconify 导出的 SVG 图标）：

```
from nicegui import ui

# 方法1：直接引用单个 SVG 文件
ui.icon('static/icons/custom-icon.svg').classes('w-10 h-10')

# 方法2：加载 SVG 图标集（通过 HTML 嵌入）
# 假设本地有 icons.svg 文件（包含多个图标，通过 symbol 定义）
ui.add_head_html('<svg xmlns="http://www.w3.org/2000/svg" style="display: none;">'
                 '<symbol id="custom-home" viewBox="0 0 24 24"><path d="M10 20v-6h4v6h5v-'
                 '8h3L12 3 2 12h3v8z"/></symbol>'
                 '<symbol id="custom-search" viewBox="0 0 24 24"><path d="M15.5'
                 '14h-.79l-.28-.27c15.41 12.59 16 11.11 16 9.5 16 5.91 13.09 3 9.5 3s3 5.91 3 9.5 5.91 16 9.5'
                 '16c1.61 0 3.09-.59 4.23-1.57l.27.28v.79l5 4.99L20.49 19l-4.99-5zm-6 0c7.01 14 5 11.99 5'
                 '9.5s7.01 5 9.5 5 14 7.01 14 9.5 11.99 14 9.5 14z"/></symbol>'
                 '</svg>')

# 引用 SVG 图标集中的图标（通过 id）
ui.icon('svg:#custom-home').classes('w-10 h-10 text-blue-500 mr-4')
ui.icon('svg:#custom-search').classes('w-10 h-10 text-green-500')

ui.run()
```

场景 5：图标动画与交互反馈

结合动画和事件，实现更丰富的交互反馈（如点击图标旋转、悬停图标缩放）：

```
from nicegui import ui

# 1. 点击旋转图标（单次旋转）
def rotate_icon(e):
    # 添加旋转动画类
    e.sender.classes('rotate-360 transition-transform duration-500')
    # 动画结束后移除类（避免重复点击叠加）
    ui.timer(0.5, lambda: e.sender.classes(remove='rotate-360 transition-transform duration-500'))

ui.icon('favorite', classes='text-2xl text-red-500 cursor-pointer').on('click', rotate_icon)

# 2. 悬停缩放图标
ui.icon('star', classes='text-2xl text-yellow-500 cursor-pointer transition-transform hover:scale-125').on('click', lambda: ui.notify('收藏成功!'))

# 3. 脉冲动画图标（用于提示）
ui.icon('notifications_active', classes='text-2xl text-orange-500 animate-pulse').tooltip('新消息提醒')

ui.run()
```

六、注意事项与优化建议

1. 图标库加载优化

- Material Icons 是默认图标库，NiceGUI 会自动加载核心资源，无需额外配置；
- Font Awesome 需手动加载资源（CDN 或本地文件），建议根据需求加载对应版本（免费版 / 精简版），避免加载冗余资源：

```
# 加载 Font Awesome 免费精简版（仅包含常用图标）
ui.add_head_html('<link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.1/css/all.min.css">')
```

- 自定义图标建议使用 SVG 格式，体积小、支持矢量缩放，避免使用 PNG（需考虑分辨率适配）。

2. 样式优先级与冲突解决

- 样式优先级：内联样式（`style` 方法）> Tailwind/Quasar 类 > 组件默认样式；
- 若图标样式未生效，检查是否存在样式冲突（如 `size` 属性与 `classes` 中的尺寸类冲突），可通过浏览器开发者工具（F12）调试；
- 建议统一使用一种样式方案（如 Tailwind 类），避免混合使用 `size`、`color` 属性与类，减少冲突。

3. 响应式适配

- 图标尺寸建议使用相对单位（如 `text-xl`、`w-10`），而非固定像素（如 `size=24`），确保在不同屏幕尺寸下适配；
- 移动端图标尺寸不宜过小（建议 $\geq 24\text{px}$ ），避免点击区域过小影响交互体验。

4. 性能优化

- 避免在页面中使用过多带动画的图标（如 `animate-spin`、`animate-pulse`），动画会占用 CPU 资源，导致页面卡顿；
- 批量使用图标时，优先使用图标库图标（Material Icons/Font Awesome），而非本地 SVG 文件（减少网络请求）；
- 自定义 SVG 图标建议精简代码（移除冗余路径、注释），减小文件体积。

5. 兼容性问题

- 部分老旧浏览器（如 IE）不支持 SVG 图标和 CSS 动画，需提供降级方案（如使用 PNG 图标）；
- Font Awesome 的部分图标（如 Pro 版图标）需付费授权，避免未经授权使用；
- 图标旋转、翻转等样式在部分低版本浏览器（如 Chrome < 60）中可能不支持，需测试兼容性。

6. 交互体验优化

- 可交互图标（如图标按钮）建议添加视觉反馈（悬停颜色变化、点击动画），提升用户体验；
- 图标尺寸不宜过大或过小，建议根据使用场景选择合适尺寸（导航栏图标：24~32px，按钮图标：16~24px）；
- 重要图标建议添加 `tooltip` 提示文本，明确图标功能（尤其对于不常用的图标）。

7. 无障碍支持

- 为图标添加 `aria-label` 属性，提升无障碍访问体验（如屏幕阅读器识别图标功能）：

```
ui.icon('search', classes='cursor-pointer').props('aria-label="搜索"')
```

- 可交互图标建议同时支持键盘操作（如 `Enter` 键触发点击事件），需结合 `on('keydown')` 事件实现：

```
ui.icon('delete', classes='cursor-pointer focus:outline-none focus:ring-2 focus:ring-red-500')\
  .on('click', lambda: ui.notify('删除'))\
  .on('keydown', lambda e: e.sender.call_method('click') if e.key == 'Enter' else None)\
  .props('tabindex="0"') # 允许键盘聚焦
```

总结

`ui.icon` 是 NiceGUI 中轻量化、灵活性强的图标组件，基于 Quasar QIcon 封装，整合了主流图标库，支持样式定制、交互事件绑定、响应式控制等核心能力。其核心优势在于与 NiceGUI 生态的无缝融合，可快速实现从简单图标展示到复杂交互元素（图标按钮、导航栏、状态提示）的各类需求。

掌握该组件的关键在于：

- 理解不同图标库的引用方式（Material Icons/Font Awesome / 自定义图标）；
- 熟练运用配置属性与样式类，定制图标外观（尺寸、颜色、动画）；
- 结合事件系统，实现图标交互逻辑（点击、悬停、状态切换）；
- 优化图标加载与性能，确保页面流畅性与兼容性；
- 结合其他组件（`ui.button`、`ui.menu`），构建统一、美观的 UI 界面。

ui.icon 全面详细阐述

`ui.icon` 是 NiceGUI 框架中用于**图标展示与交互**的轻量级核心组件，底层整合了 Quasar 图标库（Quasar Icon Set）与 Material Icons 图标库，同时支持自定义 SVG 图标、Font Awesome 图标扩展，具备样式定制、响应式适配、事件绑定等核心能力。该组件体积小、渲染快，适用于按钮图标、导航标识、状态提示、表单辅助等几乎所有网页场景，是构建简洁美观 UI 的基础组件之一。以下从核心特性、使用场景、配置选项、图标来源、进阶用法等方面展开详细说明，全程贴合官方文档核心内容，兼顾基础用法与高级扩展。

一、核心概述

1. 功能定位

- 基础能力：快速渲染预设图标（Quasar/Material）、自定义 SVG 图标，支持尺寸、颜色、样式的灵活调整；
- 交互能力：支持绑定点击、hover 等鼠标事件，可作为独立交互元素（如关闭按钮）或组件附属元素（如按钮内图标）；
- 扩展能力：兼容第三方图标库（Font Awesome、IcoMoon 等），支持自定义图标集导入，适配多场景图标需求；
- 适配能力：支持响应式尺寸、深色 / 浅色模式切换，与 Tailwind/Quasar 样式类无缝兼容，适配桌面端、移动

端。

2. 核心优势

优势点	说明
轻量化集成	内置 Quasar + Material Icons 核心图标（无需额外引入资源），渲染性能优异；
用法简洁直观	仅需传入图标名称即可使用，代码简洁，学习成本极低；
样式高度可定制	支持直接设置颜色、尺寸、旋转、阴影等，兼容所有 CSS 样式与框架样式类；
交互灵活	可独立作为交互组件（绑定点击事件），也可嵌套在按钮、卡片等组件中使用；
扩展性极强	支持自定义 SVG 图标、导入第三方图标库，无图标类型限制；
响应式适配	支持根据屏幕尺寸自动调整图标大小，适配不同设备显示需求；
生态兼容完善	与 NiceGUI 其他组件（ <code>ui.button</code> 、 <code>ui.label</code> 、 <code>ui.card</code> ）无缝联动，风格统一。

3. 核心底层依赖

`ui.icon` 的图标来源的核心是 **Quasar Icon Set**（默认集成），同时兼容 Material Icons（Quasar 内置子集），二者无需额外安装，开箱即用：

- Quasar Icon Set：NiceGUI 默认图标库，包含 1000+ 常用图标（如 `menu`、`close`、`save`、`delete`），命名简洁，适配 Quasar 组件风格；
- Material Icons：Quasar 内置的 Material Design 图标子集，图标数量更丰富，支持填充 / 轮廓两种风格（如 `home`、`home_outline`）；
- 扩展支持：通过简单配置可导入 Font Awesome、IcoMoon 等第三方图标库，或直接使用自定义 SVG 图标，突破预设图标限制。

二、基础使用（核心用法，贴合官方示例）

`ui.icon` 的基础用法极简，核心参数为 `name`（图标名称），无需复杂配置，即可快速渲染图标。以下是官方文档核心基础示例的完整拆解：

1. 基础渲染（预设图标）

直接传入 Quasar 或 Material Icons 名称，渲染默认样式图标：

```
from nicegui import ui

# 1. Quasar 预设图标（最常用）
ui.icon('menu') # 菜单图标
ui.icon('close') # 关闭图标
ui.icon('save') # 保存图标
ui.icon('delete') # 删除图标

# 2. Material Icons 图标（支持填充/轮廓风格）
ui.icon('home') # 填充风格（默认）
ui.icon('home_outline') # 轮廓风格（后缀 _outline）
```

```
ui.icon('settings')      # 填充风格
ui.icon('settings_outline') # 轮廓风格

ui.run()
```

- 关键说明：图标名称**区分大小写**（官方规范），如 `menu` 不可写为 `Menu`；部分图标有别名（如 `cancel` 等价于 `close`），可直接使用。

2. 核心基础配置（尺寸、颜色、样式）

通过 `size`、`color` 参数或 `classes` 样式类，快速定制图标外观，是日常开发最常用的配置方式：

```
from nicegui import ui

# 1. 尺寸设置（3种方式，优先级：classes > size 参数 > 默认）
ui.icon('menu', size='24px') # 直接设置尺寸（px/rem/em，推荐 px）
ui.icon('menu', size='2rem') # 响应式尺寸（适配不同屏幕）
ui.icon('menu').classes('text-3xl') # Tailwind 样式类（text-xs 到 text-9xl，最灵活）

# 2. 颜色设置（3种方式，优先级：classes > color 参数 > 默认）
ui.icon('close', color='red') # 内置颜色名（red/blue/green 等）
ui.icon('close', color='#ff0000') # 十六进制颜色
ui.icon('close', color='rgb(255,0,0)') # RGB 颜色
ui.icon('close').classes('text-red-500') # Tailwind 颜色类（最推荐，适配主题）

# 3. 基础样式优化（阴影、旋转、对齐）
ui.icon('refresh').classes('shadow-sm') # 阴影
ui.icon('refresh').classes('rotate-90') # 旋转 90 度（rotate-180/270 同理）
ui.icon('info').classes('inline-block align-middle') # 与文字垂直对齐

ui.run()
```

- 官方提示：`size` 参数的默认值为 `24px`（适配大多数组件），推荐使用 Tailwind 样式类（`text-xl` 等）进行尺寸控制，更贴合 NiceGUI 样式生态。

3. 独立交互用法

`ui.icon` 可独立作为交互组件，绑定 `click`、`hover` 等事件，替代简单按钮，适用于“关闭”“刷新”“删除”等高频交互场景：

```
from nicegui import ui

# 点击事件（最常用）
ui.icon('close', size='24px', color='red').on('click', lambda: ui.notify('点击了关闭图标'))

# hover 事件（样式切换）
icon = ui.icon('refresh', size='24px').classes('transition-all cursor-pointer')
icon.on('mouseenter', lambda: icon.classes('text-blue-500 scale-110')) # hover 放大、变色
icon.on('mouseleave', lambda: icon.classes('text-gray-500 scale-100')) # 离开恢复

# 双击事件
```

```
ui.icon('save', size='24px').on('dblclick', lambda: ui.notify('双击保存'))

ui.run()
```

三、核心配置选项（官方完整参数解析）

`ui.icon` 继承自 NiceGUI `ui.element` 组件，拥有基础组件的所有配置，同时新增图标专属配置，核心参数如下（全部来自官方文档，标注默认值与使用场景）：

属性名	类型	说明	默认值	适用场景	
<code>name</code>	<code>str</code>	图标名称（Quasar/Material 预设图标名，或自定义 SVG 图 标标识）；核心参数，必传	<code>''</code> （空字符串）	所有图标渲 染场景	
<code>size</code>	<code>`str`</code>	<code>int`</code>	图标尺寸，支持数值（自 动补 px）、字符串 (px/rem/em)；	<code>24px</code>	调整图 标大 小，适 配组件 布局
<code>color</code>	<code>str</code>	图标颜色，支持内置颜色名、 十六进制、RGB/RGBA；	<code>''</code> （继承父级）	快速设置图 标颜色，区 分状态	
<code>classes</code>	<code>Classes[Self]</code>	应用于图标元素的 CSS 类 (Tailwind/Quasar 样式 类)；	<code>''</code>	复杂样式定 制（阴影、 旋转、对 齐）	
<code>style</code>	<code>Style[Self]</code>	内联 CSS 样式（如 <code>transform: rotate(45deg);</code> ）；	<code>''</code>	临时样式调 整，优先级 高于 classes	
<code>html_id</code>	<code>str</code>	图标 DOM 元素 ID（v2.16.0+ 支持），用于 JS 操作或样式定 位；	<code>''</code>	高级定制， 需关联 JS 时使用	
<code>visible</code>	<code>BindableProperty</code>	图标是否可见，支持响应式变 量绑定；	<code>True</code>	动态控制图 标显示 / 隐 藏（如状态 切换）	
<code>props</code>	<code>`str`</code>	<code>dict`</code>	传递给底层 Quasar 图标 组件的原生属性（如 <code>flip-horizontal</code> 水平 翻转）；	<code>''</code>	原生属 性扩展 （翻 转、动 画）
<code>svg_content</code>	<code>`str`</code>	<code>None`</code>	自定义 SVG 图标内容（替 代预设图标），支持直接 传入 SVG 代码；	<code>None</code>	使用自 定义图 标，突 破预设 限制

配置示例（官方进阶配置）

```
from nicegui import ui

# 1. 原生 props 配置（翻转、动画）
ui.icon('arrow_right', props='flip-horizontal') # 水平翻转
ui.icon('refresh', props='animate-spin') # 旋转动画（加载中效果）
ui.icon('home', props='flip-vertical animate-pulse') # 垂直翻转+脉冲动画

# 2. 响应式控制显示/隐藏
show_icon = ui.reactive(True)
icon = ui.icon('info', size='24px').bind_visible(show_icon)
ui.button('切换图标显示', on_click=lambda: setattr(show_icon, 'value', not show_icon.value))

# 3. 内联 style 定制（复杂旋转、透明度）
ui.icon('alert', style='transform: rotate(30deg); opacity: 0.7;')

# 4. 自定义 SVG 图标（svg_content 参数）
ui.icon(
    name='custom-icon', # 名称可自定义（仅作为标识）
    svg_content='''
        <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" fill="currentColor">
            <path d="M12 2L2 7 10 5 10 5-10-5z"/>
            <path d="M2 17 10 5 10 5-10-5z"/>
            <path d="M2 12 10 5 10 5-10-5z"/>
        </svg>
    ''',
    size='32px',
    color='green'
)

ui.run()
```

四、图标来源详解（官方指定 4 类，全覆盖）

`ui.icon` 的图标来源分为 4 类，涵盖“预设使用 - 扩展第三方 - 自定义”全场景，官方优先推荐前两类，第三、四类用于特殊需求：

1. 内置 Quasar 图标库（默认，最推荐）

Quasar 图标库是 NiceGUI 原生集成的核心图标库，包含 1000+ 常用图标，命名简洁，适配 Quasar 组件风格，无需任何额外配置，直接使用图标名称即可。

- 核心特点：体积小、渲染快、与 NiceGUI 样式无缝兼容；
- 常用图标示例（官方高频推荐）：
 - 导航类： `menu`、`close`、`arrow_left`、`arrow_right`、`chevron_up`
 - 操作类： `save`、`delete`、`edit`、`refresh`、`download`、`upload`
 - 状态类： `info`、`warning`、`error`、`success`、`help`
 - 表单类： `search`、`check`、`radio`、`checkbox`

- 官方查询方式：可通过 Quasar 官方图标文档 (<https://quasar.dev/icons>) 查询所有可用图标及名称。

2. 内置 Material Icons 库 (扩展预设)

Quasar 内置了 Material Icons 子集，支持“填充风格”和“轮廓风格”，图标数量比 Quasar 图标库更丰富，适合需要更细腻图标风格的场景。

- 核心特点：支持双风格切换（填充 / 轮廓），图标细节更丰富；
- 使用规则：
 - 填充风格：直接使用图标名（如 `home`、`settings`、`notifications`）；
 - 轮廓风格：图标名后缀 `_outline`（如 `home_outline`、`settings_outline`）；
- 示例：

```
ui.icon('home')           # 填充风格（默认）
ui.icon('home_outline')   # 轮廓风格
ui.icon('notifications')  # 填充风格
ui.icon('notifications_outline') # 轮廓风格
```

3. 第三方图标库 (扩展用法，官方支持)

NiceGUI 支持导入第三方图标库（如 Font Awesome、IcoMoon、Bootstrap Icons），核心是通过 `ui.add_head_html` 引入第三方图标库的 CDN 资源，再通过 `ui.icon` 或 `ui.html` 渲染。

(1) Font Awesome 图标 (最常用第三方库)

Font Awesome 是目前最流行的图标库，包含免费 / 付费图标，支持多种风格（solid/regular/brands），官方文档明确支持该库的导入使用：

```
from nicegui import ui

# 1. 引入 Font Awesome 免费版 CDN (v6 版本，官方推荐)
ui.add_head_html('''
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/font-awesome/6.5.1/css/all.min.css">
''')

# 2. 渲染 Font Awesome 图标 (两种方式)
# 方式1: 使用 ui.icon, name 格式为 "fa-[风格]-[图标名]"
ui.icon('fa-solid fa-coffee', size='24px', color='brown') # solid 风格 (填充)
ui.icon('fa-regular fa-user', size='24px')                # regular 风格 (轮廓)
ui.icon('fa-brands fa-github', size='24px', color='black') # brands 风格 (品牌图标)

# 方式2: 使用 ui.html (更灵活，支持更多属性)
ui.html('<i class="fa-solid fa-heart text-red-500 text-2xl"></i>')
```


(2) 其他第三方图标库 (IcoMoon/Bootstrap Icons)

```
from nicegui import ui

# 1. 引入 Bootstrap Icons CDN
ui.add_head_html('''
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-
icons@1.11.3/font/bootstrap-icons.min.css">
''')

# 渲染 Bootstrap Icons
ui.icon('bi-bootstrap', size='24px', color='blue')

# 2. 引入 IcoMoon CDN (需先在 IcoMoon 生成自定义图标库)
ui.add_head_html('''
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/icomoon-
icons@1.0.0/style.css">
''')
ui.icon('icon-home', size='24px')
```

4. 自定义 SVG 图标 (终极扩展, 官方重点支持)

当预设图标和第三方图标无法满足需求时, 可通过 `svg_content` 参数直接传入 SVG 代码, 实现完全自定义的图标, 支持任意形状、颜色、细节定制, 无样式限制。

核心要求 (官方规范)

1. SVG 代码需包含 `xmlns="http://www.w3.org/2000/svg"` 命名空间 (必传, 否则无法渲染) ;
2. 建议设置 `viewBox="0 0 24 24"` (与预设图标尺寸一致, 适配布局) ;
3. 填充色推荐使用 `currentColor` (继承 `ui.icon` 的 `color` 参数, 方便统一控制颜色) ;

示例 (官方自定义 SVG 案例)

```
from nicegui import ui

# 自定义 SVG 图标 (箭头+文字组合)
ui.icon(
    name='custom-arrow',
    svg_content='''
        <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" fill="currentColor">
            <path d="M5 12h14M12 5l7 7 7 7"/>
            <text x="12" y="22" font-size="8" text-anchor="middle"
fill="currentColor">Go</text>
        </svg>
    ''',
    size='32px',
    color='blue',
    classes='cursor-pointer'
).on('click', lambda: ui.notify('点击了自定义图标'))

# 自定义纯色 SVG 图标
ui.icon(
```

```

name='custom-circle',
svg_content=''
    <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24">
        <circle cx="12" cy="12" r="10" fill="currentColor"/>
    </svg>
'',
color='red',
size='24px'
)

```

五、高频使用场景（官方示例 + 实际开发场景）

场景 1：作为按钮附属图标（最常用）

与 `ui.button` 嵌套使用，增强按钮的视觉辨识度，符合网页设计规范，官方文档优先推荐该用法：

```

from nicegui import ui

# 1. 图标在按钮左侧（默认）
ui.button('保存', icon='save', on_click=lambda: ui.notify('保存成功'))

# 2. 图标在按钮右侧（通过 props 配置）
ui.button('删除', icon='delete', props='icon-right', color='red')

# 3. 纯图标按钮（无文字，节省空间）
ui.button(icon='close', size='sm', color='gray').on('click', lambda: ui.notify('关闭'))
ui.button(icon='refresh', size='sm', color='blue').on('click', lambda: ui.notify('刷新'))

# 4. 按钮组+图标（导航栏常用）
with ui.button_group():
    ui.button(icon='arrow_left', size='sm')
    ui.button(icon='pause', size='sm')
    ui.button(icon='arrow_right', size='sm')

```

场景 2：导航栏 / 侧边栏图标

用于导航菜单、侧边栏，搭配文字使用，提升导航的视觉体验，适配管理系统、移动端页面：

```

from nicegui import ui

# 侧边栏导航（图标+文字）
with ui.sidebar().classes('w-64'):
    ui.label('主导航').classes('font-bold text-lg mb-2')
    # 导航项（图标+文字，hover 高亮）
    ui.button('首页', icon='home', classes='w-full justify-start mb-1').on('click', lambda:
ui.notify('进入首页'))
    ui.button('设置', icon='settings', classes='w-full justify-start mb-1')
    ui.button('消息', icon='notifications', classes='w-full justify-start mb-1')
    ui.button('退出', icon='logout', color='red', classes='w-full justify-start')

```

场景 3：状态提示图标

用于表单验证、操作反馈、系统状态提示，快速传递信息，无需文字说明：

```
from nicegui import ui

# 1. 表单验证状态
with ui.column().classes('gap-2'):
    ui.label('表单提交状态').classes('font-bold')
    ui.row([ui.icon('success', color='green'), ui.label('提交成功')])
    ui.row([ui.icon('error', color='red'), ui.label('提交失败，请重试')])
    ui.row([ui.icon('warning', color='orange'), ui.label('存在警告，请注意')])
    ui.row([ui.icon('info', color='blue'), ui.label('请填写必填项')])

# 2. 系统状态提示（悬浮显示详情）
ui.icon('help', size='20px', color='gray').tooltip('点击查看帮助文档')
```

场景 4：表单辅助图标

用于输入框、下拉框等表单组件，提示组件功能（如搜索、清空、选择），提升表单交互体验：

```
from nicegui import ui

# 1. 搜索输入框（前置搜索图标）
with ui.input(placeholder='请输入关键词').classes('w-64') as input:
    with input.add_slot('prepend'):
        ui.icon('search', color='gray').classes('mr-2')

# 2. 输入框（后置清空图标）
with ui.input(placeholder='请输入内容').classes('w-64') as input:
    with input.add_slot('append'):
        ui.icon('close', color='gray', classes='cursor-pointer').on('click', lambda:
input.clear())

# 3. 下拉框（前置图标）
with ui.select(['选项1', '选项2', '选项3'], placeholder='请选择').classes('w-64') as select:
    with select.add_slot('prepend'):
        ui.icon('arrow_down', color='gray')
```

场景 5：动态图标（响应式控制）

结合 NiceGUI 响应式变量，动态切换图标、颜色、尺寸，适配状态变化（如加载中、在线 / 离线）：

```
from nicegui import ui

# 1. 动态切换图标（在线/离线）
is_online = ui.reactive(True)
icon = ui.icon(
    name='check_circle' if is_online.value else 'cancel',
    color='green' if is_online.value else 'red'
)
```

```
# 绑定响应式变量，状态变化时自动更新
icon.bind_name_from(is_online, lambda val: 'check_circle' if val else 'cancel')
icon.bind_color_from(is_online, lambda val: 'green' if val else 'red')
ui.button('切换状态', on_click=lambda: setattr(is_online, 'value', not is_online.value))

# 2. 加载中动画图标（动态旋转）
ui.icon('refresh', props='animate-spin', size='24px', color='blue').tooltip('加载中...')
```

六、进阶技巧（官方扩展用法 + 性能优化）

1. 图标动画（官方支持的原生动画）

通过 `props` 参数使用 Quasar 内置动画，无需额外 CSS，适用于加载中、提示等场景：

```
from nicegui import ui

# 1. 旋转动画（加载中）
ui.icon('refresh', props='animate-spin', size='24px', color='blue')

# 2. 脉冲动画（提示）
ui.icon('warning', props='animate-pulse', size='24px', color='orange')

# 3. 呼吸动画（强调）
ui.icon('success', props='animate-breath', size='24px', color='green')

# 4. 自定义动画（结合 classes）
ui.icon('heart', classes='text-red-500 animate-beat', size='24px')
```

- 官方提示：内置动画无需额外配置，仅支持 Quasar 预设动画（`animate-spin`/`animate-pulse`/`animate-breath`），自定义动画需通过 Tailwind 样式类实现。

2. 图标组合与叠加

通过 `ui.container` 嵌套多个图标，实现组合图标效果，适用于复杂标识场景：

```
from nicegui import ui

# 组合图标（通知图标+数字角标）
with ui.container().classes('relative inline-block'):
    ui.icon('notifications', size='24px', color='gray')
    # 数字角标（叠加在图标右上角）
    ui.label('3').classes('absolute -top-1 -right-1 bg-red-500 text-white text-xs rounded-full w-5 h-5 flex items-center justify-center')
```

3. 响应式尺寸适配

结合 Tailwind 响应式样式类，实现不同屏幕尺寸下的图标尺寸自适应，适配桌面端、移动端：

```
from nicegui import ui

# 小屏（手机）：text-sm（16px），中屏（平板）：text-base（20px），大屏（电脑）：text-xl（24px）
ui.icon('home').classes('text-sm md:text-base lg:text-xl')

# 响应式颜色（深色/浅色模式）
ui.icon('moon').classes('text-gray-700 dark:text-gray-300')
```

4. 性能优化（官方建议）

- 优先使用内置 Quasar/Material 图标：无需额外加载资源，渲染速度最快，避免过度依赖第三方图标库；
- 自定义 SVG 图标优化：简化 SVG 代码（删除冗余路径、注释），统一 `viewBox` 尺寸（24x24），避免过大 SVG 导致渲染卡顿；
- 避免图标过度嵌套：减少图标容器的层级嵌套，避免同时渲染大量图标（如列表中每个项都有图标时，尽量复用样式）；
- 懒加载第三方图标库：若仅在特定页面使用第三方图标，可通过 `ui.add_head_html` 动态引入，避免初始加载冗余资源。

5. 图标样式优先级（官方规范）

样式生效优先级从高到低：

1. `style` 内联样式（如 `style='color: red;'`）；
 2. `classes` 样式类（Tailwind/Quasar 类，如 `text-red-500`）；
 3. `color/size` 参数；
 4. 父级元素继承样式；
- 官方建议：优先使用 `classes` 进行样式定制，便于样式统一管理和响应式适配。

七、注意事项（官方警告 + 常见问题）

1. 图标名称大小写敏感：Quasar/Material 图标名称严格区分大小写（如 `menu` 不可写为 `Menu`、`MENU`），错误命名会导致图标无法渲染；
2. 第三方图标库引入顺序：导入第三方图标库（如 Font Awesome）时，需先引入 CDN 资源，再渲染图标，否则图标无法显示；
3. 自定义 SVG 命名空间必传：`svg_content` 中的 SVG 代码必须包含 `xmlns="http://www.w3.org/2000/svg"`，否则渲染失败；
4. 图标尺寸适配：预设图标默认尺寸为 `24px`，自定义 SVG 图标建议设置相同 `viewBox`（24x24），避免与其他组件布局错位；
5. 深色模式适配：若使用深色模式，建议通过 `dark:text-*` 样式类设置图标颜色，避免图标在深色背景下不可见；
6. 动画性能：避免给大量图标同时添加动画（如 `animate-spin`），会导致页面卡顿，仅在必要场景使用；
7. 图标兼容性：内置图标兼容所有现代浏览器（Chrome/Firefox/Edge/Safari），第三方图标库和自定义 SVG 需注意老旧浏览器（如 IE）兼容性。

总结

`ui.icon` 是 NiceGUI 中轻量化、高灵活度的核心图标组件，核心价值在于“开箱即用的预设图标 + 无限扩展的自定义能力”，完美适配从简单 UI 到复杂交互场景的所有图标需求。其底层依托 Quasar 与 Material 图标库，无需额外配置即可快速使用，同时支持第三方图标库导入和自定义 SVG 图标，兼顾易用性与扩展性。

掌握该组件的关键在于（贴合官方文档核心）：

1. 熟练掌握内置图标（Quasar/Material）的使用，记住高频图标名称；
2. 灵活运用 `color` / `size` / `classes` 进行样式定制，适配页面布局；
3. 掌握第三方图标库（Font Awesome）和自定义 SVG 图标的导入与使用；
4. 理解图标与其他组件（按钮、表单、导航）的联动用法；
5. 规避常见问题（命名错误、SVG 配置错误、样式优先级），优化渲染性能。